

# Sistemas Distribuídos — Aula 32

*Encerramento do Semestre: Devolutivas e Fechamento de Notas · 17/12 · Prof. Leandro Borges*

## Da Aula 31 à Aula 32: o Que Muda Hoje

A Aula 31 foi a apresentação final dos trabalhos: cada grupo mostrou, ao vivo, a versão completa do sistema distribuído construído ao longo do semestre — do Checkpoint 1 (comunicação cliente-servidor) ao Checkpoint 3 (tolerância a falhas), passando pela replicação e pela discussão de consistência do Checkpoint 2. Hoje não há apresentação nem conteúdo técnico novo. A Aula 32 é a última aula do semestre e tem um propósito diferente: fechar o ciclo. Isso significa três coisas — dar uma devolutiva geral sobre o que foi visto nas apresentações, explicar exatamente como a nota final de cada um será composta e onde consultá-la, e fazer um passo atrás para olhar o semestre inteiro, do primeiro conceito de sistema distribuído até o projeto funcionando na prática.

Esta leitura segue a mesma estrutura da aula: primeiro a devolutiva sobre os trabalhos, depois o fechamento de notas, em seguida a retrospectiva do semestre e sua conexão com o mercado de trabalho, e por fim uma mensagem de encerramento com sugestões para quem quiser continuar estudando o tema.

## Devolutiva Geral — Pontos Fortes das Apresentações

De forma geral, os grupos desta turma evoluíram bem ao longo do bimestre, e isso ficou evidente na Aula 31. Quatro pontos se destacaram na maioria das apresentações. Primeiro, arquiteturas bem justificadas: os grupos souberam explicar por que escolheram sockets, RPC ou RMI para o domínio do projeto escolhido — um sistema de reservas, um chat, um marketplace simplificado — em vez de apenas implementar a primeira opção que funcionasse. Segundo, replicação acompanhada de discussão real sobre consistência: a maioria não se limitou a duplicar dados entre réplicas, mas discutiu conscientemente os trade-offs entre consistência forte e consistência eventual, retomando exatamente o conteúdo da Aula 20.

Terceiro, e talvez o ponto mais valioso, vários grupos demonstraram cenários de falha de verdade, ao vivo — um nó caindo durante a apresentação e o sistema reagindo — em vez de apenas descrever esse comportamento no relatório. Esse tipo de demonstração é o que separa um sistema que "parece" tolerante a falhas de um sistema que de fato foi testado sob falha, tema central das Aulas 21 a 23. Quarto, os relatórios finais tornaram visível a evolução entre os três checkpoints: dava para acompanhar, checkpoint a checkpoint, o que mudou e por quê — exatamente a proposta do modelo de projeto evolutivo adotado nesta disciplina, inspirado em cursos de projeto de universidades como MIT e Stanford.

## Desafios Comuns e Sua Conexão com os Temas da Disciplina

Nenhum dos pontos a seguir foi eliminatório para nenhum grupo — mas vale a pena nomeá-los, porque cada um deles aponta exatamente para um dos temas centrais que estudamos ao longo do semestre, e reconhecê-los é útil tanto para quem for revisar o próprio projeto quanto para quem for trabalhar com sistemas distribuídos profissionalmente depois desta disciplina.

**Confundir replicação com simples "cópia de segurança":** alguns grupos implementaram réplicas de dados sem definir claramente como elas permaneceriam consistentes entre si durante o uso normal do sistema — o problema de consistência de dados discutido na Aula 20 não desaparece só porque existem múltiplas cópias.

**Servidor único como ponto de gargalo e de falha:** em alguns projetos, apesar de haver réplicas de dados, ainda existia um único ponto de entrada não redundante — contrariando diretamente as metas de escalabilidade e a lógica de tolerância a falhas vistas nas Aulas 3 e 21 a 23.

**Cliente fortemente acoplado ao endereço fixo do servidor:** quando o cliente precisa saber exatamente onde o servidor está, o sistema perde a transparência de localização — uma das metas de projeto centrais apresentadas já na Aula 3, logo no início do semestre.

**Ausência de testes de falha real, só descrita "no papel":** relatórios que afirmam que o sistema "tolera a queda de um nó", sem nenhuma evidência de um teste de fato executado, deixam a detecção e a recuperação de falhas — Aulas 21 a 23 — como uma promessa não verificada.

**Pouca discussão sobre concorrência entre réplicas:** quando duas réplicas podem ser atualizadas ao mesmo tempo, entra em jogo o sincronismo e a exclusão mútua distribuída vistos nas Aulas 6 e 7 — um ponto que vários grupos implementaram sem discutir explicitamente no relatório.

O padrão é claro: cada desafio comum de projeto encontrado nos trabalhos remete diretamente a um dos grandes temas estudados no semestre. Isso não é coincidência — é exatamente o que a disciplina se propõe a ensinar: os conceitos teóricos existem porque evitam armadilhas reais de engenharia, e os trabalhos são o lugar onde essas armadilhas aparecem de fato.

## Fechamento de Notas: Como a Média Final é Composta

A fórmula da média final é a mesma definida desde o início do semestre e não muda agora: Média Final =  $0,4 \times \text{Prova 1 (P1)} + 0,6 \times \text{Trabalho Final}$ . A prova P1 avaliou o conteúdo mais teórico do 1º bimestre, enquanto o Trabalho Final — que vale a maior parte da nota — acompanhou e avaliou a evolução do projeto prático ao longo de todo o semestre.

A nota do Trabalho Final, por sua vez, é a soma ponderada dos três checkpoints somados à participação. O Checkpoint 1 — comunicação cliente-servidor funcional, código organizado e relatório claro — vale 25% da nota do trabalho. O Checkpoint 2 — replicação implementada com discussão de consistência coerente com o conteúdo da disciplina — vale 30%. O Checkpoint 3 — tolerância a falhas implementada, relatório final completo e a apresentação realizada na Aula 31 — vale 35%, a maior fatia entre os checkpoints, já que é onde o projeto chega à sua forma mais completa. Por fim, a participação — a contribuição individual de cada integrante dentro do grupo, avaliada por autoavaliação e por observação em aula — vale os 10% restantes.

Vale reforçar: a nota da apresentação da Aula 31 não é uma nota separada e independente — ela está embutida na linha do Checkpoint 3, junto com a implementação da tolerância a falhas e o relatório final entregue.

## Devolutiva Individual e Consulta de Notas

Cada grupo recebe, além da nota, um retorno detalhado: a nota de cada checkpoint (1, 2 e 3) separadamente, feedback escrito sobre a arquitetura, a estratégia de replicação e o tratamento de falhas implementados, e observações específicas sobre a apresentação feita na Aula 31. Esse feedback tem valor além da nota em si — é um registro do que funcionou bem e do que poderia ser melhorado num próximo projeto de sistemas distribuídos, seja acadêmico ou profissional.

As notas finais são lançadas no portal acadêmico da Uni-FACEF em até alguns dias após o encerramento do semestre. Dúvidas sobre a nota ou pedidos de revisão devem ser encaminhados por e-mail institucional — não é necessário aguardar uma nova aula presencial para isso, já que esta é a última do semestre. O código-fonte e o relatório de cada grupo permanecem como referência de portfólio: é um projeto real, com comunicação em rede, replicação e tolerância a falhas implementadas do zero, que vale a pena manter disponível para mostrar em processos seletivos futuros.

## **Retrospectiva do Semestre: o Arco do Aprendizado**

Vale a pena, nesta última aula, olhar para o semestre inteiro de uma vez e enxergar o arco que ele desenhou. O 1º bimestre (Aulas 1 a 13) construiu os fundamentos: o que caracteriza um sistema distribuído frente a um sistema centralizado, paralelo ou de alto desempenho; as metas de projeto que orientam qualquer decisão de arquitetura — transparência, flexibilidade, confiabilidade, desempenho e escalabilidade; a comunicação entre processos e o sincronismo, com relógios lógicos e físicos, exclusão mútua e algoritmos de eleição; e, por fim, os modelos concretos de comunicação — cliente-servidor com sockets, RPC, RMI, serviço de grupo e arquiteturas peer-to-peer.

O 2º bimestre (Aulas 14 a 31) elevou o nível de complexidade. Começou com computação em grid, cluster e computação móvel — sistemas que agregam poder computacional ou lidam com nós que mudam de localização. Passou pelo núcleo teórico mais denso do curso: os algoritmos distribuídos de consenso, snapshot global e replicação de dados, seguidos pela tolerância a falhas — modelos de falha, detecção, redundância e recuperação — e pelos sistemas operacionais distribuídos, que sustentam tudo isso na prática, da gerência de processos aos sistemas de arquivos distribuídos. E fechou com o trabalho final, onde cada grupo aplicou essa teoria toda construindo, de fato, um sistema distribuído funcional em grupo.

O fio condutor de ponta a ponta é simples de enunciar, mesmo que difícil de dominar por completo: um sistema distribuído precisa parecer, para quem o usa, um único sistema coerente — apesar de ser, por baixo, um conjunto de máquinas independentes, sujeitas a atrasos de rede, a relógios que não estão perfeitamente sincronizados e a falhas parciais que podem acontecer a qualquer momento. Cada tema do semestre — transparência, consistência, sincronismo, tolerância a falhas — é uma resposta a algum aspecto dessa tensão entre a ilusão de um sistema único e a realidade de múltiplos nós independentes.

## **Sistemas Distribuídos no Mercado de Trabalho**

Os conceitos vistos neste semestre não são exercício teórico isolado: eles aparecem, com outros nomes, no dia a dia de quem trabalha com infraestrutura de TI, backend ou dados. A computação em nuvem — AWS, Azure, Google Cloud — é, na prática, a meta de escalabilidade da Aula 3 vendida como serviço: provisionar mais capacidade sob demanda é justamente lidar com o crescimento do sistema sem reescrever sua arquitetura. A arquitetura de microsserviços, hoje padrão em boa parte das empresas de tecnologia, é comunicação entre processos aplicada em escala — cada serviço é, essencialmente, um nó independente que troca mensagens com os demais via RPC ou APIs REST, ecoando diretamente o conteúdo das Aulas 8 a 10.

Bancos de dados distribuídos como Cassandra, DynamoDB ou CockroachDB colocam em produção, todos os dias, exatamente a discussão de replicação e consistência da Aula 20 — a escolha entre consistência forte e eventual não é um detalhe acadêmico, é uma decisão de arquitetura com impacto direto no produto. Ferramentas de mensageria e streaming, como Kafka e RabbitMQ, implementam a comunicação por eventos e o serviço de grupo vistos na Aula 10. E a orquestração de containers com Kubernetes automatiza, em grande escala, exatamente o failover e a

tolerância a falhas discutidos nas Aulas 21 a 23 — quando um nó falha, o orquestrador detecta e substitui, sem intervenção humana.

Até a prática de SRE (Site Reliability Engineering) e observabilidade — hoje uma área de carreira consolidada em empresas de tecnologia — é, essencialmente, aplicar continuamente os conceitos de detecção de falhas e sistemas resilientes vistos nesta disciplina. Quem entende por que um sistema distribuído falha, e como ele se recupera, tem uma vantagem real ao entrar em qualquer time que opere sistemas em produção.

## Encerramento e Sugestões para Continuar Estudando

Foi um semestre de bastante conteúdo — da definição básica de um sistema distribuído até algoritmos de consenso, tolerância a falhas e sistemas operacionais distribuídos, fechando com um projeto em grupo que colocou tudo isso em prática. A expectativa é que a forma de pensar em transparência, consistência e resiliência construída ao longo destas 32 aulas continue útil muito além desta disciplina — seja em outra matéria do curso, em um estágio, ou no primeiro emprego em tecnologia.

Para quem quiser ir além do que foi visto em sala, três caminhos valem a pena. Como curso, o MIT 6.5840 — Distributed Systems (disponível gratuitamente como OpenCourseWare, com laboratórios práticos em Go) é a referência mais citada mundialmente sobre o tema; a especialização Cloud Computing, da Universidade de Illinois, oferecida na Coursera, também aprofunda a aplicação prática em nuvem. Como leitura, o livro Designing Data-Intensive Applications, de Martin Kleppmann, é hoje uma das obras mais recomendadas sobre sistemas distribuídos de dados em produção, e o livro Site Reliability Engineering, do Google, está disponível gratuitamente online e conecta diretamente tolerância a falhas com a prática de operar sistemas reais. Como comunidade, vale acompanhar a ACM SIGOPS e as conferências SOSP e OSDI, cujos artigos e palestras são abertos ao público, além de grupos de leitura de artigos técnicos como o "Papers We Love", presente em diversas cidades e também online.

Foi um prazer lecionar para esta turma ao longo do semestre. Bom trabalho a todos, e sucesso nos próximos passos da carreira.

## Referências

TANENBAUM, A. S.; STEEN, M. V. Sistemas Distribuídos: princípios e paradigmas. 2ª ed. São Paulo: Pearson Prentice Hall, 2008.

COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T. Sistemas distribuídos: conceitos e projeto. 4ª ed. Porto Alegre: Bookman, 2007.

KUROSE, J. F.; ROSS, K. W. Redes de Computadores e a Internet. 5ª ed. Addison Wesley, 2010.

COMER, D. E. Redes de Computadores e Internet. 4ª ed. Bookman, 2007.

KLEPPMANN, M. Designing Data-Intensive Applications. O'Reilly Media.

GOOGLE. Site Reliability Engineering. O'Reilly Media (disponível gratuitamente em [sre.google/books](https://sre.google/books)).

Trabalho — Sistema Distribuído Evolutivo: Briefing e Rubrica, material entregue no início do semestre desta disciplina.